SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE CODE: CSA14 **COURSE NAME: COMPILER DESIGN**

COURSE OUTCOME COVERED

CO2: *Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation (BL3)*

Assessment Tool Weightage: 20%

Duration :60

Mins

On Class

QUESTIONS: 25 **Total**
Marks:50

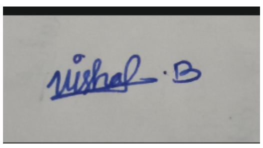
Annexure A

SHORT ANSWER TEST

Code of Conduct:

I am VISHAL B(192524485) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Annexure A

SHORT ANSWER TEST

Q1. A lexical analyzer produces the following token stream:
id + id
Explain how the parser validates the syntactic structure of the expression.

Q2. Given the grammar:
$S \rightarrow aS \mid b$
Determine whether the string aaab belongs to the language using leftmost derivation.

Q3. Remove left recursion from the grammar:
$A \rightarrow Aa \mid b$

Q4. Find FIRST(S) for the grammar:
$S \rightarrow aA \mid b$
$A \rightarrow c \mid \varepsilon$

Q5. Determine whether the grammar is suitable for top-down parsing.
$S \rightarrow Sa \mid b$
Justify your answer.

Q6. Apply left factoring to the grammar:
S → if E then S
S → if E then S else S

Q7. Construct a parse tree for the string:
a + b
using the grammar:
$E \rightarrow E + E$
$E \rightarrow id$

Q8. Write the recursive descent procedures for the grammar:
$S \rightarrow aA$
$A \rightarrow b$

Q9. Given the grammar:
$S \rightarrow AB$
$A \rightarrow a$
$B \rightarrow b$
Construct the predictive parsing table.

Q10. Demonstrate the first three actions of a shift-reduce parser for the input:
id + id

Q11. Identify the handle in the string:
id + id
for the grammar:
$E \rightarrow E + E$
$E \rightarrow id$

Q12. Construct the operator precedence relations for the operators:

+
*
where * has higher precedence than +.

Q13. Explain how operator precedence parsing handles the expression:
id * id

Q14. What is an LR parser? Illustrate the parsing process using the grammar:
S → aA
A → b

Q15. Given the grammar:
S → aA
A → b
Write the steps involved in constructing an SLR parsing table.

Q16. Compute FOLLOW(A) for the grammar:
S → AB
A → a
B → b

Q17. Construct the augmented grammar for:
S → a

Q18. Find the closure of the LR(0) item:
S' → •S
for the grammar:
S → a

Q19. Check whether the grammar is ambiguous:
E → E + E
E → id
using the string:
id + id + id

Q20. Construct a leftmost derivation for the string:
id + id
using the grammar:
E → E + E
E → id

Q21. Apply left factoring to the grammar:
S → aA
S → aB
A → b
B → c

Q22. Find FIRST(A) for the grammar:
A → x
A → y

Q23. Determine whether the grammar is suitable for predictive parsing.
S → aA | aB
A → b
B → c
Justify your answer.

Q24. Construct recursive descent parser procedures for the grammar:
S → aS
S → b

Q25. Build the predictive parsing table for the grammar:
S → aS
S → b
Show the table entries for the terminals:
a, b, $

Q24. Construct recursive descent parser procedures for the grammar:
S → aS
S → b

Q25. Build the predictive parsing table for the grammar:
S → aS

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

Name : B.vishal

Reg no : 192524485

S → aA
A → b

Q15. Given the grammar:
S → aA
A → b
Write the steps involved in constructing an SLR parsing table.

Q16. Compute FOLLOW(A) for the grammar:
S → AB
A → a
B → b

Q17. Construct the augmented grammar for:
S → a

Q18. Find the closure of the LR(0) item:
S' → •S
for the grammar:
S → a

Q19. Check whether the grammar is ambiguous:
E → E + E
E → id
using the string:
id + id + id

Q20. Construct a leftmost derivation for the string:
id + id
using the grammar:
E → E + E
E → id

Q21. Apply left factoring to the grammar:
S → aA
S → aB
A → b
B → c

Q22. Find FIRST(A) for the grammar:
A → x
A → y

Q23. Determine whether the grammar is suitable for predictive parsing.
S → aA | aB
A → b
B → c
Justify your answer.

Q24. Construct recursive descent parser procedures for the grammar:
S → aS
S → b

Q25. Build the predictive parsing table for the grammar:
S → aS
S → b
Show the table entries for the terminals:
a, b, $

SHORT ANSWER

TEST

Name : B.vishal

Reg NO : 192524485

Course Code : CSA

Course Name : complier Design.

1. id+id :

a. Parser checks whether "id+id" follows the grammer and constructs or forms the Parse tree.

2. $S \to aS | b$

$L =$ aS
aaS
aaaS
aaab

$\therefore$ Yes, the string aaab belongs to the Language using leftmost derivation.

3. $A \to Aa | b$:

General form : $S \to a\alpha / \beta$

after removing

$A \to bA'$

$A' \to aA' | \varepsilon$

4. $S \to aA | b$

$A \to c | \varepsilon$

first (S) = {a, b}

first (A) = {c, ε}

first (B) = {b}

5. $S \to Sa|b$

No, it has direct Left recursion $S \to Sa$

LL(1) and recursive descent parser's cannot handle Left recursion, remove

it first.

$S \to bS'$
$S' \to aS' | \epsilon$

6. $S \to$ if E then S
$S \to$ if E then S else S

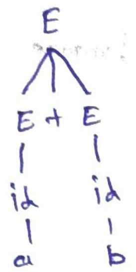
In, this common factor is " if E then S "

after applying left factoring grammar,

$S \to$ if E then S S'
$S \to$ else S | ϵ

7. a+b parse tree:

$E \to E + E$
$E \to id$



$E \to E+E \to id+id \to a+b$.

8. $S \to aA$
$A \to b$

recursive descent procedure.

```
S()
{
 match(a);
 A()
}
A()
{ match(b);
}
```

S()
⇓
match (a)
⇓
A()
⇓
match (b)
⇓
accept

9. $S \to AB$
$A \to a$
$B \to b$

Predicative parsing table.

First (A) = {a}
First (B) = {b}
First (S) = {a}

Non-terminal	a	b	$
S	$S \to AB$	Error	Error
A	$A \to a$	Error	Error
B	Error	$B \to b$	Error

10. id + id

Step	stack	Input	Action
1	id	id $	shift
2	E	+ id $	Reduce ($E \to id$)
3	E+	id$	shift

11. Handle string
id+ id

Grammar,
$E \to E+E$
$E \to id$
∴ It first Handle id

12. Operator +, *

Relation
+ < *
* > +
+ = +
* = *

	+	*
+	>	<
*	>	>

13. Expression id * id.

Parser actions : shift id
Reduce id → t
shift *
conflit *
shift id
Reduce id → E
Reduce E*E → E
Accept

14. LR parson :

∴ LR parser means reduce input Left to right most derivativation as reverse.

parsing ab.

shift +a

shift +b

Reduce $A \to b$

∴ Grammar :

$S \to aA$

$A \to b$

15. $S \to aA$

$A \to b$

SLR parsing table.

∴ Scans Input Left to right and produces Rightmost derivation reverse.

$ab \to$ shift $a \to$ shift $b \to$ Reduce $b \to A$ $\to$ Reduce $aA \to S$ $\to$ Accept.

16. $S \to AB$

$A \to a$

$B \to b$

Follow $(A) = \{b\}$

17. $S \to a$

Augmented grammar.

$S' \to S$

$S \to a$

18. $S' \to \cdot S$

for closure

$S \to \cdot a$

19. Ambiguous or not.

∴ It is Ambiguous because id + id has

(id + id) + id and id + (id + id)

20. ~~E~~ ~~E → id +~~ $E \to E+E$

$E \to id$

leftmost derivation

$E \to E+E$

$id + E$

$id + id$

21. S → aA

S → aB

A → b

B → c

S → aS'

~~A → b~~ S' → A | B

A → b

B → c

22. A → x

B → Y

First(A) = {x, y}

23. No. Both productions begin with a causing a parsing conflict.

24.
```
S()
{
  if (next== a);
  {
    match(a);
    S();
  }
  else
    match(b);
}
```

25. S → aS

S → b

Non-terminal	a	b	$
S	S → aS	S → b	error